Contents

Known bugs and v1.0 limitations

v1.0 known limitations (workarounds documented; v1.1 candidates)

These are not bugs in the strict sense — they're scope decisions for v1.0 that we may revisit based on real-world testing.

`.dta` round-trip loses `xtset` state

`save mydata.dta` then `use mydata.dta` preserves variable types, formats, variable labels, and value labels, but does **not** preserve the `xtset` panel/time variable assignment. Stata writes this as a dataset characteristic (`_dta[iis]`, `_dta[tis]`); readstat doesn't expose characteristic writing in its public API, and hand-writing characteristic blocks in raw .dta format is a v1.1 task.

Workaround: re-run `xtset country year` after `use`. This is the standard Stata-do-file convention anyway, so most users won't notice.

ARIMA uses conditional likelihood, not exact ML (no Kalman filter)

tea's `arima` command computes parameters by minimizing the SSR of the recursively-computed residuals, conditioning on ε_{<0}=0 (the conditional likelihood approach). This gives consistent estimates but slightly different finite-sample behavior than Stata's default exact ML via the Kalman filter:

- Absolute log-likelihood values differ from Stata's (we drop the initial-state contribution)
- AR coefficients near the unit-root boundary may converge less robustly
- No seasonal ARIMA (sar, sma) in v1.0

Workaround: for serious time-series work, escape to R's `arima()` or Python's `statsmodels.tsa.arima`.

Factor variable composition with TS ops

`i.country#c.L.gdp` (factor interaction with a lagged regressor) is **not** supported directly. The factor module operates on column names, not on materialized TS-op temps.

Workaround: pre-compute the lag manually: `gen L1_gdp = L.gdp`, then use `i.country#c.L1_gdp`.

`xtreg, be` doesn't weight by T_i

Stata's default `xtreg, be` uses a slight variance correction for unbalanced panels (down-weights groups with few observations). tea uses straight OLS on the panel means. For balanced panels the answers are identical. For unbalanced panels they can differ by a few percent in the SEs.

MA(1) and ARMA SE precision

ARIMA standard errors are computed from a numerical-differentiation Hessian (forward differences scaled by 1e-5). For models with q>0 (MA components), the SEs can be off by 1-5% compared to analytical or exact-ML Hessians. Point estimates are unaffected.

Variable-name length limit (32 chars)

A residual Stata-12 limit. Factor-variable interaction names like `2010.year#1985.country#c.gdp` (28 chars) fit; longer 3-way interactions truncate. In practice not a problem.

No graphics

tea ships zero plotting commands. Users wanting plots should `export delimited` and pipe to `gnuplot`, R's `ggplot2`, or Python's `matplotlib/seaborn`.

CHANGELOG (recently fixed)

Recently fixed

Tier 4 — econometric completion (NEW for v1.0)

`ivregress 2sls` (in `src/regress.c` ~300 lines):

Syntax: `ivregress 2sls y [exog] (endog = instruments) [, vce(robust|cluster v)]`. Full first-stage regression of each endogenous regressor on the combined instrument matrix W = [exog, Z, _cons] via dgels, second-stage $\beta=(\hat{X}'\hat{X})^{-1}\hat{X}'y$; residuals computed using ORIGINAL X (not fitted) for SE; sandwich V = A·B·A for robust/cluster ($A=(\hat{X}'\hat{X})^{-1}$, B uses original-X residuals). HC1 / CR1 finite-sample adjustments. First-stage F-stat reported (minimum across endogenous regressors — the weak-instrument worry case).

Verified on textbook endogeneity case: with cov(x, u) ≠ 0, OLS biased to 2.49; IV recovers 1.997 against true 2.0, first-stage F=1364.

`poisson` — count outcomes via the existing MLE driver:

Added `mle_family_poisson` to src/mle.c with `poisson_per_obs`: μ=exp(η) clipped at η<700; score=y-μ; weight=μ; loglik=yη-μ (drops the log(y!) constant — absolute loglik differs from Stata but β identical). Added `count_validate` (rejects y<0). Generalized do_glm_binary header text to "Poisson regression". Verified intercept-only: β_cons=log($\bar{y}$)=0.6931 for $\bar{y}$=2 (exact).

`xtreg, be` (between estimator):

Replaced the "not implemented" stub. After group-mean computation (already done in the FE/RE path), branch into BE: build G_obs × (K_orig+1) matrix Xb (panel means + appended 1s column for cons), OLS via dgels, σ^2_be=RSS/(G-K), $V=\sigma^2\cdot(\bar{X}'\bar{X})^{-1}$ via Cholesky. Verified hand-checkable case: panel data with $\bar{y}_i = 5 + 2\bar{x}_i$ recovers β_x=2, _cons=5, R^2_b=1.0000 exactly.

`arima` (new module `src/arima.c`, ~580 lines):

Conditional-likelihood ARIMA(p,d,q) with optional ARIMAX exog regressors. Syntax: `arima y [exog], arima(p d q) [noconstant]`. Algorithm: - Difference y in place d times - Pack params as [μ, β_K_ex, φ_p, θ_q] - Recursive residual computation with ε_{<0}=0 conditional assumption - Gauss-Newton iteration with step halving (max 50 iterations) - SSR via finite-difference gradient + Hessian - $V \approx 2\sigma^2\cdot\text{Hess}^{-1}$ via dgetrf/dgetri with ridge

Verified: AR(1) on strong-signal data recovers φ=0.7010 (true 0.7). AR(1) on US WEO growth data matches OLS lag regression to 3 digits ($\hat{\varphi}$ = 0.154 both). MA(1) Monte Carlo with rnormal() noise: θ=0.429 (true 0.5) at N=500.

`runiform()` and `rnormal()` (in eval.c):

PCG32-based PRNG with Box-Muller normal. `set seed N` for reproducibility. Essential for Monte Carlo, simulation, and bootstrap workflows. Tested against expected distributional moments: mean 0.492, sd 0.291 for u (vs true 0.5, 0.289); mean 0.001, sd 1.015 for z (vs true 0, 1).

Tier 3 — output and reproducibility (NEW)

`estimates` command suite (new module `src/estcmd.c`):

```
estimates store  NAME      clone last_est, save under name
estimates restore NAME     load NAME → last_est (for predict/margins/test)
estimates dir              list saved with cmd, N, depvar
estimates drop NAME...     remove named entries; `_all` clears
estimates table NAMES      compact side-by-side comparison table
                           (options: se, t, p, star, stats(N r2 rmse F ...))
```

Aliased as `est`. Workspace gains a linked-list field `stored_est` (in `dataset.h`); freed by `ws_free`. Built on the existing `est_clone` infrastructure from the hausman work.

`estout` — publication-ready tables (new module `src/estout.c`):

```
estout [names] [using FILE] [, format(latex|markdown|md|plain)
                               se|t|p stars stats(...) nogaps
                               title("...") label(tab:foo)
                               keep(varlist) drop(varlist)
                               nomtitles mtitles("M1" "M2" ...)]
```

Default format is LaTeX (per the IMF / academic-economics convention). LaTeX output produces a `tabular` block with significance stars in the superscript style (`$^{***}$`), variable names properly escaped, and a footnote legend. Stata-style `\_cons` for the constant term. Markdown and plain text variants also supported.

Verified end-to-end on a three-model comparison with the standard options: stars apply z-distribution thresholds (10/5/1%), SEs go in parentheses below each coef, stats footer rows handle N/r2/r2_a/rmse/F/ df_r/df_m/sigma_u/sigma_e/rho.

`log` audit and fix:

The `log` command captures output via a printf-macro override in `commands.c`. Problem: `display` lives in `interp.c` and was using bare `printf`, so its output went to stdout but not the log file. Fixed by making `g_logfp` extern and adding a manual tee from `do_display`. Round-trip verified: `log using file.log` now captures both command echoes (`. display ...`) AND their output (`_b[x] = 2`).

`.dta` round-trip audit:

Spot-checked save → use preservation: - Variable types: □ - Variable formats (`%8.2f`, `%td`): □ - Variable labels (`label variable x "GDP per capita"`): □ - Value labels (`label define`, `label values`): □ - Numeric values: □ (down to floating-point precision) - String columns: □

Not preserved in `.dta`: `xtset` panel/time variable assignment. Stata writes this as a dataset characteristic (`_dta[iis]`, `_dta[tis]`); readstat doesn't expose characteristic writing in its public API, and hand-writing the characteristic block in raw .dta format is a v1.1 project. Workaround for v1.0: re-run `xtset country year` after `use`, which is also the standard Stata-do-file convention.

Tier 2 — factor variables: i., c., #, ##, ib. (NEW)

The Stata factor-variable grammar, now supported across every estimator that goes through `tsop_expand_varlist` (regress, xtreg fe/re, logit, probit, predict, margins). Lives in new module `src/factor.[ch]`.

Grammar:

```
i.var           dummies for each non-base level (base = smallest by default)
ib<n>.var       same, but base = level n
c.var           explicit continuous (display prefix; mostly for use in #)
A#B             interaction: cross product of A's and B's columns
A##B            main effects + interaction: equivalent to "A B A#B"
```

Examples (assume country has levels {1,2,3}, year has {2010,2011,2012}):

```
i.country           → 2 cols: "2.country" "3.country"
ib2.country         → 2 cols: "1.country" "3.country"
i.country#c.gdp     → 2 cols: "2.country#c.gdp" "3.country#c.gdp"
i.country#i.year    → 4 cols: "2.country#2011.year" "2.country#2012.year"
                              "3.country#2011.year" "3.country#2012.year"
i.country##c.gdp    → 5 cols total: 2.country, 3.country, gdp,
                                    2.country#c.gdp, 3.country#c.gdp
```

Architecture:

- `factor.c` is a sibling to `tsop.c`: tokens are parsed into atoms (kind 'I' for `i.`, 'C' for `c.`, 'L' for coef-name "2.x", or bare variable in a # context). Atoms join into terms via #. ## is unrolled to the power-set of single-term products (main effects + all interactions).
- `factor_is_factor_token` is the dispatch test; `factor_expand_token` produces a snapshot-able list of temp frame columns.
- Plumbed into `tsop_expand_varlist`: each token is tried as a TS-op first; if that returns "not mine," tried as a factor-var; if that returns "not mine," falls through to plain `varlist_expand`.

Predict/margins re-materialisation (the subtle bit):

When `predict` sees a coefficient named `2.country` or `2.country#c.gdp`, the column doesn't exist anymore — it was a temp dropped after the estimator finished. Solution: treat coef-name form `<level>.<varname>` as a new atom kind 'L' (level-pinned indicator). When `tsop_expand_varlist` is called with `"2.country"`, it dispatches to factor, which produces a single column with values 1{country==2}. Same mechanism handles `2.country#c.gdp` (indicator × continuous).

Verified:

Spec	Hand-checkable result
`regress y x i.country` on y = α_c + x	β=1, _cons=α_1, 2.country=α_2-α_1, …
`regress y x ib2.country`	_cons=α_2, 1.country=α_1-α_2
`regress y c.x#c.x` on y = x^2	coef = 1.0 exactly
`regress y i.country##c.x` on y = α_c + β_c·x	reproduces all α and β exactly
`regress growth L.growth i.cid` (LSDV) on WEO	matches `xtreg fe` exactly

`predict` after `logit y x i.g` now works (was the open bug from the previous session). `margins, dydx(*)` after factor-var estimation works.

Restrictions in v1.0:

- `i.var` requires `var` to be numeric. Use `encode` for string ids.
- Composition with TS-ops (e.g., `i.country#c.L.gdp`) is not supported. Manually pre-compute the lag (`gen L1_gdp = L.gdp`) and use `c.L1_gdp`.
- Cell-count cap of 10000 columns per token (huge interactions blocked).

- Variable-name length limit of 32 chars means very long interaction names truncate. In practice not a problem for typical use.

Tier 1 polish — statistical functions, egen extensions, _b/_se, formats

A pass over the everyday-use surface in preparation for v1.0. An audit showed that tea's expression-language function library was already very comprehensive (math, trig, hyperbolic, strings, dates, distributions, glob+regex, value/variable labels, encode/decode/destring/tostring, recode, format). The gaps that mattered:

New statistical functions (in `src/eval.c`): - `normalden(x)` / `normalden(x,σ)` / `normalden(x,μ,σ)` — normal PDF - `lnnormal(x)` — log Φ(x), stable for large negative x via the asymptotic expansion (reuses `tea_log_normal_cdf`) - `lnnormalden(x)` — log φ(x) closed-form

Better regex (in `src/eval.c`): - `regexs(n)` — extract n-th submatch from the last `regexm` call. Uses a file-scope `tea_regex_submatch[20]` buffer. Index 0 is the whole match; 1..N are capture groups. - `regexr(s, pat, rep)` — replace first regex match with a string

Extended egen (in `src/commands.c`): - New aggregators: `median`, `iqr`, `p25`, `p75`, `p50`, `pc` (with option `p(N)` to specify percentile), `rank`, `tag` - Multi-argument row functions now work: `rowtotal(v1 v2 v3)`, `rowmean`, `rowmin`, `rowmax`, `rowsd`, `rowmiss`, `rownonmiss` (the previous implementation was single-arg only) - `tag(g1 g2)` marks the first row of each group as 1, others 0 - `rank(x)` uses average-rank-for-ties

Format application (in `src/commands.c::fmt_cell`): Custom printf-style numeric formats set via `format var %w.dF` are now honored by `list` (previously only date formats like `%td` were).

Coefficient/SE macros after estimation (in `src/regress.c` + `src/interp.c`):

`_b[name]` and `_se[name]` are now accessible everywhere `e(N)` is. The estimators (regress, xtreg, logit, probit) store macros keyed by `_b[varname]` and `_se[varname]` (including bracketed names like `_b[L.growth]` for TS-op regressors). `macro_expand` recognises the pattern and substitutes outside double quotes; inside quotes the text stays literal. Standard Stata behaviour.

This unlocks idioms like:

```
quietly regress y x
display "t-stat = " _b[x] / _se[x]
gen yhat = _b[_cons] + _b[x]*x
```

Bug fix — quoted-string macro substitution (in `src/interp.c`): `display "e(N) = " e(N)` used to produce `"3 = 3"` because the macro expander substituted both occurrences of `e(N)`, including the one inside the quoted string. Fixed by tracking `in_dquote` state and skipping `e()` / `r()` / `_b[]` / `_se[]` substitution inside strings. Stata's behaviour preserved: backtick-locals and `$globals` are still substituted inside strings, since those are Stata's quote-aware substitution syntax.

`margins` — marginal effects after regress / logit / probit (NEW)

```
margins, dydx(*)              AME for all continuous regressors
margins, dydx(x1 x2)          AME for specified regressors
margins, dydx(*) atmeans      MEM (effects at sample means)
margins, dydx(x1)             AME for x1 only
```

After OLS, the AME of x_k is exactly β_k and the SE is exactly SE(β_k) — the margins output reproduces the coefficient table. Useful as a sanity check.

After logit / probit, AMEs are the economically meaningful number: how the predicted *probability* changes with x, averaged across the sample. Coefficients themselves aren't directly interpretable (they're log-odds for logit, latent-index units for probit).

Math:

Logit: m_k = (1/N) Σ_i Λ(X_iβ)·(1-Λ(X_iβ)) · β_k Probit: m_k = (1/N) Σ_i φ(X_iβ) · β_k OLS: m_k = β_k

Delta-method SE: V(m_k) = G_k · V(β) · G_k', where G_k is the K-vector ∂m_k/∂β_j = (1/N) Σ_i [g'(η_i) X_ij β_k + g(η_i) 1{j=k}]. Uses the full V(β) matrix from the previous estimation (so robust/cluster SEs flow through automatically).

Verified: - Logit and probit on the same data produce nearly identical AMEs (a known result — they only differ at the tails, which contribute little to the average). In one test case AME(logit, x1) = 0.12237, AME(probit, x1) = 0.12253 — different in the fifth decimal. - AME for OLS reproduces the regress coefficient table exactly. - TS-op coefficient names (`L.x` etc.) work via the same snapshot-then-drop trick `predict` uses.

Deferred for v2: - `margins x1` (predicted outcomes at varying x1 — useful for plots) - `margins, at(x1=(0 1) x2=10)` (predictions at specified values) - Discrete-change effects for indicator variables - Marginal effects after xtreg (would need within-effect-aware formulas)

`predict` extended to logit, probit, and xtreg (NEW)

The old `predict` worked only after regress, with options `xb` and `residuals`. Rewritten to dispatch on the previous estimator's `cmd`:

```
After regress:
  predict yhat            xb (default)
  predict r, residuals    y - Xβ
  predict s, stdp         standard error of the linear prediction

After logit / probit:
  predict p               predicted probability (default)
  predict idx, xb         linear index Xβ

After xtreg (fe or re):
  predict yhat            xb (default; does NOT include u_i)
  predict u, u            α̂_i  (panel effect)
  predict e, e            idiosyncratic residual y - Xβ - α̂_i
  predict ue, ue          α̂_i + e  (= y - Xβ)
  predict xbu, xbu        Xβ + α̂_i  (in-sample prediction)
```

The dispatch lives in `predict_resolve_kind` and the per-row evaluation is one big switch. Sensible errors when options don't match the preceding command (e.g. `predict p, pr` after regress fails cleanly).

TS-op coefficient names: when `e->xnames[j]` is `L.growth` (TS-op form), the column doesn't exist in the frame anymore at predict time, but we can re-materialize it via the shared `tsop_expand_varlist`. A subtlety: we have to **snapshot** the materialized data into our own buffer, drop the temps, THEN add the new prediction variable — otherwise `tsop_drop_temps` would drop the just-added prediction column instead of the temp. This

is what an earlier draft got wrong (a regression test would have shown a list-not-found error).

xtreg specifics: α̂_i is computed in one pass over the in-sample rows (`e->used` tells us which) by streaming through the panel-sorted data and accumulating ȳ − x̄'β per panel. For predictions on rows that weren't in the estimation sample, α̂ is unknown so `u/e/ue/xbu` return missing.

Verified on the hand-checkable FE case (3 panels × 4 obs, perfect fit) where the math is exact: β=1, α=10/20/30, e=0, xbu=y. Also tested on WEO AR(1) growth where predict reproduces the fit yhat + r = growth exactly.

`logit` and `probit` — binary-outcome MLE (NEW)

```
logit  y x1 x2 ... [if] [in] [weight], [vce(robust|cluster v)]
probit y x1 x2 ... [if] [in] [weight], [vce(robust|cluster v)]
```

Implemented via a Newton-Raphson MLE driver in `src/mle.[ch]` with family-specific callbacks for the score, weight, and log-likelihood contributions. Same driver should extend to poisson, cloglog, etc. with no changes to the iteration logic — just new MleFamily structs.

Output matches Stata's layout: iteration log, LR χ^2, pseudo R^2, coefficient table with z-stats, optional cluster adjustment note.

Standard errors: - (default): classical (X'WX)^{-1} (asymptotic inverse-Hessian) - `vce(robust)`: HC1 sandwich - `vce(cluster v)`: CR1 cluster-robust on score

Numerical care: - log(1+e^η) computed stably as max(η,0) + log(1+e^{-|η|}) — no overflow. - Probit's φ/Φ uses asymptotic Mills bound λ(η) → -η when Φ(η) underflows. Per-obs Fisher weight clipped at 1e8. - Starting values: OLS solve, which also detects rank deficiency (collinear regressors marked omitted, held at 0 through iteration). - Step halving (up to 8 halvings) when full Newton step decreases □. - Perfect-separation detected when □ → 0; warning printed.

Verified: - Intercept-only on 50/50 sample: β_cons = 0 exactly. SE = 0.2 for logit, 0.1253 for probit (matches closed forms). - Real-effect data: classical β_logit/β_probit ≈ 1.81 ratio reproduced. - Newton converges in 3-5 iterations for well-conditioned data.

Postestimation: `test` and `lincom` work on logit/probit estimates. (test uses F with df_r = N-K; the asymptotic test would be χ^2, but the difference is negligible for typical sample sizes.)

`test` syntax extended (NEW)

The `test` command now supports the full Stata-style hypothesis syntax:

```
test var                    H0: β_var = 0
test var = 0                same, explicit
test var = 0.5              H0: β_var = 0.5 (numeric RHS)
test var1 = var2            H0: β_var1 = β_var2 (equality)
test var1 var2 ...          joint H0: all listed = 0
```

Internally constructs the restriction matrix R and value vector r, then computes the Wald F = (Rβ - r)' (R V R')^{-1} (Rβ - r) / q against F(q, df_r). Pre-existing `test L.growth L2.growth` (joint zero) is a special case and still works.

`lincom` accepts TS-op coefficient names (NEW)

The lincom parser now treats `.` as part of a coefficient name when followed by a name character, so `lincom L.growth + L2.growth` works correctly. Previously the parser stopped at the first `.` and tried to look up "L" as a coefficient.

`xtreg, re` — random-effects (GLS) estimator (NEW)

Implements panel RE via feasible GLS with quasi-demeaning:

```
xtreg y x1 x2 ... [if] [in], re [vce(robust|cluster v)]
```

Method: the model is `y_it = α_i + x_it'β + ε_it` with α_i ~ (0, σ_u²) assumed independent of x_it. The estimator transforms each variable by subtracting θ_i times the panel mean, where:

```
θ_i = 1 - σ_e / √(T_i · σ_u² + σ_e²)
```

Then runs OLS on (y, *X*, 1-θ_i) — note an explicit constant column, unlike FE. σ_e² comes from the within (FE) regression; σ_u² comes from the variance components computation (max(0, var(α̂_i) - σ_e²/T̄)). The transform interpolates between OLS (θ=0) and FE (θ=1).

Output matches Stata's xtreg, re layout: - "Random-effects GLS regression" header - R-within, R-between, R-overall — all computed with β̂_RE - σ_u, σ_e, ρ — variance components (shared with FE math) - θ — quasi-demeaning factor; min/avg/max for unbalanced panels - _cons coefficient and SE (unlike FE) - Wald χ² for the slopes - corr(u_i, X) = 0 noted as the RE assumption

Verified on: - Degenerate hand-checkable case (perfect within fit → θ=1 → RE collapses to FE; β_RE = β_FE = 1.0 exactly; _cons omitted since the 1-θ column is identically zero). - WEO AR(1) growth: β_RE,L = 0.245 sits between pooled OLS and FE β = 0.220. σ_u/σ_e/ρ = 0.97/5.37/0.032 — most variance within- panel. θ varies 0.19-0.39 across 210 countries.

v1 limitations: - Uses σ_u² = max(0, σ²_BE - σ_e²/T̄) with T̄ = N/n for unbalanced. Stata's exact Swamy-Arora uses a slightly different T̄ correction; for balanced panels the estimators agree.

`hausman` — FE vs RE specification test (NEW)

```
xtreg y x1 x2 ..., fe
xtreg y x1 x2 ..., re
hausman
```

Computes H = (β_FE - β_RE)' [V_FE - V_RE]^{-1} (β_FE - β_RE), distributed χ²(K_common) under H₀: cov(α_i, x_it) = 0. Rejection means RE is inconsistent; use FE.

Tea saves the last xtreg fe and last xtreg re into dedicated workspace slots automatically, so the user just types `hausman` with no arguments after running both. Order doesn't matter.

Implementation notes: - Only compares slope coefficients present (and not omitted) in both estimates. `_cons` is dropped since FE doesn't have one. - The difference matrix V_FE - V_RE should be PSD under H₀, but sampling noise (especially with robust/cluster SEs) can make it not strictly PD. Cholesky inverse is tried first; if it fails, falls back to an SVD pseudoinverse. When the resulting statistic comes out negative, we report |H| with a "sign flipped" warning. - Verified on WEO AR(1) growth: χ²(1) = 189, p < 0.0001 → strongly rejects RE (matches the FE-reported corr(u_i, Xb) = 0.97 warning sign). On a constructed case where α_i ⊥ x by construction, χ² ≈ 0 and the test correctly fails to reject.

xtreg, fe — fixed-effects (within) estimator (NEW)

Implements panel FE OLS via within transformation:

```
xtreg y x1 x2 ... [if] [in], fe [vce(robust|cluster v)]
```

Reports R-within, R-between, R-overall, σ_u, σ_e, ρ, corr(u_i, Xβ), and the F-test that all u_i = 0. Verified on a hand-checkable case (`tests/regression/29_xtreg_fe.do`): 3 panels × 4 obs, y = α_i + x with α ∈ {10,20,30} gives β_FE = 1.0 exactly, R-within = 1.0, σ_u = 10 exactly. Verified against the WEO AR(1) growth dynamic showing internally consistent results (β from `xtreg D.growth L.growth, fe` = -(1 - β from `xtreg growth L.growth, fe`)).

Standard error options: | `, fe` | classical: σ²·(X_w'X_w)^{-1} with df = N - n_groups - K | | `, fe vce(robust)` | HC1 sandwich on within-transformed data | | `, fe vce(cluster v)` | CR1 cluster-robust on within-transformed data |

v1 limitations (deferred to v2): - `_cons` is not displayed. Its point estimate (mean of α_i) is computable but the strictly correct SE requires LSDV-equivalent variance propagation we haven't implemented. Users wanting inference on _cons can run `regress y x i.panel` for now. - Stata defaults `vce(robust)` to cluster-by-panel for xtreg fe; tea's `vce(robust)` is HC1. Use explicit `vce(cluster panelvar)` for cluster-by-panel. - `xtreg, be` (between effects) is not yet implemented — returns a clean error suggesting `, fe` or `, re`. (`, re` ships in this version — see above.) - Weights work the same way as in `regress` (build_design handles them), but the FE-specific weight semantics have not been carefully verified yet.

TS-op handling unified across all varlist-accepting commands (NEW)

Time-series operator support is now centralized in `src/tsop.[ch]`. The parser handles every Stata form once (`L.x`, `L2.x`, `L(1/2).x`, `L.(x y)`, `L2.(x y)`, `L(1/2).(x y)`, step form `L(1(2)9).x`), and every command that accepts a varlist routes through `tsop_expand_varlist`. Concretely:

Command	TS ops	Mechanism
regress	✓	tsop_expand_varlist (depvar + xspec)
summarize	✓	tsop_expand_varlist
list	✓	tsop_expand_varlist
tabulate	✓	tsop_expand_varlist
tabstat	✓	tsop_expand_varlist
collapse	✓	tsop_expand_varlist (paren-aware tokenizer)

`regress` lost ~400 lines of duplicated parser code in this refactor (from 1198 down to 796 lines). Future commands that accept varlists — xtreg, ivregress, correlate, etc. — get TS-op support for free by calling `tsop_expand_varlist`. Test 28 exercises the universal coverage; tests 21–27 cover individual command surfaces.

Side fix discovered: `keep if` / `drop if` used to call `frame_unsort` which over-eagerly cleared `ts_panel` / `ts_time`. Row deletion doesn't invalidate panel structure (the columns are still there and still sorted; gaps are fine for tea's gap-aware tsidx lookup), so the ts state now survives those operations.

xtdescribe / xtdes (NEW)

Added `xtdescribe` (with `xtdes` abbreviation) to summarize the panel structure declared by `xtset`. Shows number of panels, time range, delta, span, min/p25/p50/p75/max distri-

bution of obs per panel, and balanced/unbalanced determination. Test 27 exercises both balanced and unbalanced cases.

TS operators in `summarize` and `list` (FIXED)

`summarize L.x` and `summarize D.x` failed with "variable not found" because those commands called `varlist_expand` directly and didn't recognize TS-op tokens. The earlier TS-op fix only touched `regress`.

Fixed by adding a shared `tsop` module (`src/tsop.[ch]`). Both `summarize` and `list` now route their varlist through `tsop_expand_varlist`, which materializes TS-op tokens as temporary frame columns (with canonical display names like `L.x`, `D2.y`) and appends them to the frame for the duration of the command, then drops them via `tsop_drop_temps` before returning. Test 26 covers this.

D-operator iterated differencing (FIXED — silent math bug)

`D2.x`, `D3.x`, etc. all returned the same as `D.x` because the eval code hardcoded `back = -1` for any D-kind operator. This was a **silent** numerical bug: regressions including `D2.x` would compute correctly for `D.x` but the higher-order differences were all collinear with `D.x` and got "omitted" — looking like a model specification issue rather than a math error.

Stata's convention is that `D#.x` is the iterated `#`-th difference:

- `D.x  = x[t] - x[t-1]`
- `D2.x = D(D.x) = x[t] - 2·x[t-1] + x[t-2]`
- `D3.x = x[t] - 3·x[t-1] + 3·x[t-2] - x[t-3]`
- `D^k x[t] = Σ_{j=0..k} (-1)^j · C(k,j) · x[t-j]`

Fixed in `src/eval.c` using a binomial-recurrence loop. The `S` operator (separate semantics: simple gap-aware seasonal difference, `S#.x = x[t] - x[t-#]`) is unchanged. Test 25 verifies on `x = t²` where `D.x` is linear, `D².x` is constant 2, `D³.x` is constant 0.

Action item for users: any analysis using `D2.x`, `D3.x`, or higher in past sessions silently produced wrong values. Re-run any such code.

Operator-list TS syntax (FIXED)

Stata's compact form `regress y L(1/2).x` (expanding to `L1.x L2.x`) was rejected with "independent variables not found". Tea now handles the full set of Stata TS-op forms in regress varlists:

Syntax	Expands to
`L.x`	`L.x`
`L2.x`	`L2.x`
`L0.x`	x (no shift)
`L(1/3).x`	`L.x L2.x L3.x` (range)
`L(1 3).x`	`L.x L3.x` (explicit list)
`L(1(2)9).x`	`L.x L3.x L5.x L7.x L9.x` (step form)
`L.(x y)`	`L.x L.y`
`L2.(x y)`	`L2.x L2.y`
`L(1/2).(x y)`	`L.x L.y L2.x L2.y` (cross product)

Implemented in src/regress.c via the unified try_tsop_form parser plus parse_numlist (range / explicit list / step) and find_matching_paren for nested parens. The regressor tokenizer is paren-aware so spaces inside (...) don't break tokenization. Tests 21, 23, and 24 cover the forms.

Multi-digit time-series operators read wrong lag (FIXED)

L2.x was actually computing the 12th lag instead of the 2nd; L3.x the 13th, F2.x the 12th lead, S3.x a 13-period seasonal difference. Affected expressions, regress, and any other consumer.

The bug was in try_tsop in src/parse.c: the lag-magnitude accumulator was initialized to 1 instead of 0, so reading "L2" gave num = 1*10 + 2 = 12. Lag-1 (L.x / L1.x / no digits) accidentally worked because num stayed at 1 either way.

Fixed by initializing num = 0. The existing if (!have) num = 1; fallback still gives bare L. the right value of 1. Test 22 pins down correct multi-digit operator behavior.

Time-series operators rejected in regress varlist (FIXED)

regress growth L.growth if country_id == "USA" failed with "regress: independent variables not found" because the regress code called varlist_expand directly on each regressor token; that helper only knew about plain names, wildcards, ranges, and _all — not the L.x / F.x / D.x / S.x syntax.

Fixed by refactoring build_design in src/regress.c to detect TS-op tokens and route them through the existing expression evaluator (expr_parse → N_TSOP). Plain names still go through varlist_expand. Works for both the dependent variable and the regressors (regress D.growth L.growth is supported). Test 21 covers regress with TS operators.

capture and _rc (FIXED in v1.4.0)

Fixed: capture now suppresses all output of the captured command (stdout and stderr, at the file-descriptor level) and _rc returns the captured return code — 0 on success, the error code otherwise — exactly as in Stata. One small remaining deviation: a successful command that is NOT under capture leaves _rc unchanged rather than resetting it to 0; branch on _rc only immediately after a capture.

test 08 captures fewer lines than expected

The reshape-error regression test passes, but if you read the expected output you'll notice some error messages print to stderr while others to stdout, and the line ordering across the merged stream depends on line-buffering. This is OK as of v0.5.20 because we set the stdout buffer to line-buffered in main.c, but the harness assumes the ordering is stable — if a future change reorders error/output emission between commands, the test may need updating.

Storage types are display-only

gen byte x = 1 accepts the byte qualifier but stores the value as double internally; gen str10 country = "USA" accepts str10 but doesn't track width. This is a deliberate design decision documented in COMPATIBILITY.md — tea is double + string internally regardless of declared type, and the .dta writer auto-compresses on save. Not a bug.

Numeric function family coverage

Many Stata numeric functions are not yet implemented:

- Distribution functions: `normal`, `normalden`, `invnormal`, `chi2`, `chi2tail`, `invchi2`, `t`, `ttail`, `invttail`, `F`, `Ftail`, `binomial`, `binomialp`, `poisson`, `poissonp`.
- Matrix functions (we don't have a matrix type yet).
- Random-number generators: `runiform`, `rnormal`, etc. (Tied to `set seed`.)

The existing function set covers basic arithmetic, transcendentals, date/time, string manipulation, and the common aggregator functions used in `egen`.

v0.6 .dta polish (this milestone)

- **Value labels** — `label define` / `label values` sets are now written into `.dta` and read back via `use`. Round-trips cleanly (test 16).
- **Dataset label** — `label data "text"` now persists. Visible in `describe`, round-trips through save/use (test 17).
- **`save FILE.dta, version(NUM)`** — emit any DTA format 104-119 (test 18). Defaults to 118 (Stata 14).
- **Workspace label-set cleanup** — `clear` and `use, clear` now also drop the workspace's value-label sets, matching Stata semantics.

Still deferred from v0.6

- **Notes** — Stata's `note: text` on a dataset or variable. Not yet read or written.
- **strL strings > 2045 bytes** — currently strings longer than the str# limit are truncated at 2045 on write. Real-world risk is essentially zero for econ data; will add when needed.
- **`.dta` fweight metadata writes are silently dropped** — readstat 1.1.9 accepts `readstat_writer_set_fweight_variable` but does not serialize the resulting Stata `_fweight` characteristic. Tea's read-side captures the metadata correctly when present (so real Stata-saved files round-trip through tea correctly *if* the user doesn't save through tea), and the write-side wiring is in place for when readstat fixes this. Track upstream.

reported by user during real-data testing

- Bug 1 — _N in display context returned 1 → **FIXED in v0.5.20**
- Bug 2 — tabstat columns() ignored in by-mode → **FIXED in v0.5.19**
- Bug 3 — tabstat with if + by showed empty by-groups → **FIXED in v0.5.18**
- Bug 4 — list * segfaulted on 147-var dataset → **FIXED in v0.5.14**
- Bug 5 — quoted filename in import not opened → **FIXED in v0.5.12**

reported by Claude during synthetic testing

- Bug 6 — commas in string literals broke parser → **FIXED in v0.5.20**
- Bug 7 — $1 in strings expanded as undefined macro → **FIXED in v0.5.20**
- Bug 8 — egen silently skipped strings → **FIXED in v0.5.20**